Tree

A **Tree** is a **non-linear data structure** that represents data in a **hierarchical** form.
It consists of **nodes** connected by **edges**, where each node stores data and references to its children.

A tree has the following properties:

- There is one **root node** (topmost node).
- Each node may have **zero or more child nodes**.
- There is exactly **one path** between the root and any other node.

Mathematical / Graph-Theoretic Definition

A **tree** is a **connected acyclic graph**, i.e.,
a graph **G = (V, E)** that satisfies:

- **Connectivity:** There is exactly one path between any two vertices.
- **Acyclic:** The graph contains **no cycles**.

Thus,
for a tree with **n vertices**,
the number of edges **E = n – 1**.

Basic Terminology

Term	Description	Example
Node	Each element of the tree	A, B, C, D
Root	Topmost node	A
Parent	A node having child nodes	A is parent of B, C
Child	Nodes that descend from a parent	B and C are children of A
Leaf Node	Node with no children	D, E
Edge	Link between two nodes	A–B
Level	Distance from the root (root is level 0)	B and C are level 1
Height	Longest path from root to a leaf	For A→B→D, height = 2
Subtree	A tree formed by any node and its descendants	B-D-E forms a subtree

Structure of a Node (C/C++ Representation)

```
struct Node {
    int data;
    Node *left;
    Node *right;
};
```

- `data` → stores value.

- `left` and `right` → point to child nodes.

Types of Trees

(a) General Tree

A **each node can have any number of child nodes** (not limited to two).
It is the most **generic** form of tree structure and serves as the foundation for more specialized trees like **binary trees**, **n-ary trees**, etc.

Example: Family Tree

```
        Grandfather
        /     |      \
   Father   Uncle    Aunt
   /  \        \
Son  Daughter  Cousin
```

- "Grandfather" = **Root**
- "Father", "Uncle", "Aunt" = **Children of root**
- "Son", "Daughter", "Cousin" = **Leaves**

Applications

Application	Example
Organization hierarchy	CEO → Managers → Employees
File directory system	C:\ → Folder → Subfolder → File
XML / HTML DOM structure	Tags nested within other tags
AI / Decision trees	Decision → Outcomes

(b) Binary Tree

A **Binary Tree** is a **hierarchical data structure** where each node can have **at most two children** — known as:

- **Left child**
- **Right child**
  A Binary Tree is a tree data structure in which each node has at most two child nodes, referred to as the left child and the right child.

Structure

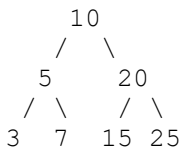
Each node has:

- **Data** field.
- **Left pointer** (points to left child).
- **Right pointer** (points to right child).

C++ Structure:

```
struct Node {
    int data;
    Node* left;
    Node* right;
};
```

Example



- Root = 10
- Left Subtree = (5, 3, 7)
- Right Subtree = (20, 15, 25)

Properties of Binary Tree

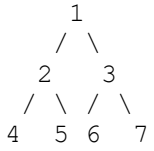
Property	Description
Max children	At most 2 per node
Empty tree	A tree with no nodes
Root node	Topmost node
Height (h)	Longest path from root to leaf
Nodes in level l	Maximum nodes = 2^l
Max nodes in tree of height h	(2^(h+1)) − 1

Types of Binary Trees

1. Full Binary Tree

A binary tree is **Full** if every node has **either 0 or 2 children** — no node has only one child.

Example:



Nodes (2, 3) each have 2 children.
Leaves (4,5,6,7) have 0 children.

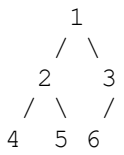
Property:
If there are *n* leaf nodes, total nodes = **2n – 1**

2. Complete Binary Tree

A binary tree is **Complete** if:

- All levels are completely filled,
- Except possibly the last level,
- And the last level has all nodes as **left as possible**.

Example:



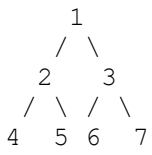
Last level not full but filled **left to right**.

3. Perfect Binary Tree

A binary tree is **Perfect** if:

- All internal nodes have **two children**, and
- All leaves are at the **same level**.

Example:



Properties:

- Total nodes = 2^(h+1) – 1
- Leaf nodes = 2^h
- Height = $\log_2(n + 1) – 1$

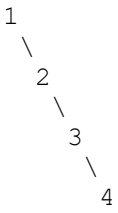
4. Skewed Binary Tree

A **Skewed Tree** is a binary tree in which **all nodes have only one child**, making it look like a **linked list**.

Two types:

- **Left Skewed Tree:** All nodes have left child only.
- **Right Skewed Tree:** All nodes have right child only.

Example (Right Skewed):



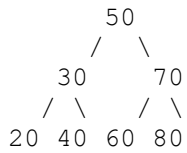
Height = n – 1,
and traversal takes O(n) time.

(c) Binary Search Tree (BST)

A **special binary tree** where:

- Left subtree has nodes with **smaller** values.
- Right subtree has nodes with **greater** values.

Example:



Properties:

- Searching, insertion, deletion – all take **O(h)** time (where h = height of tree).
- Balanced BST gives **O(log n)** time.

(d) AVL Tree

- A **self-balancing BST**.
- For every node, the **height difference** (balance factor) between left and right subtrees ≤ 1.

(e) B-Tree / B+ Tree

- Used in databases and file systems.
- Each node can have **multiple keys** and **multiple children**.
- Keeps data **sorted** and supports efficient **insertion/deletion/search**.

Operations

- **Insertion**
- **Deletion**
- **Traversal (Preorder, Inorder, Postorder, Level-order)**
- **Searching**
- **Height/Depth calculation**

Tree Traversal Methods

Traversal means **visiting every node** in a tree **exactly once** in a specific order.
It helps in performing operations like **displaying data, searching, copying, or deleting nodes**.

In tree traversal, we systematically visit nodes either:

- **Depth-wise** (going deep into the tree first), or
- **Breadth-wise** (visiting all nodes level by level).

There are mainly **two categories** of tree traversal:

1. **Depth First Traversal (DFS)**
2. **Breadth First Traversal (BFS)**

(A) Depth First Traversal (DFS)

Depth First Search traverses the tree **as deep as possible along each branch before backtracking**.

It uses **recursion** or **stack**.

DFS has three standard types for binary trees:

1. **Inorder Traversal (Left → Root → Right)**
2. **Preorder Traversal (Root → Left → Right)**
3. **Postorder Traversal (Left → Right → Root)**

1. Inorder Traversal (Left → Root → Right)

Algorithm Steps:

1. Traverse the **left subtree**.
2. Visit the **root node**.
3. Traverse the **right subtree**.

Recursive Algorithm:

```
void inorder(Node* root) {
    if (root != NULL) {
        inorder(root->left);
        cout << root->data << " ";
        inorder(root->right);
    }
}
```

Example Tree:

Traversal Steps:

```
Left → Root → Right:
4 → 2 → 5 → 1 → 3
```

Output: `4 2 5 1 3`

Use Case:
In **Binary Search Tree (BST)**, **Inorder traversal** gives **sorted order** of elements.

2. Preorder Traversal (Root → Left → Right)

Algorithm Steps:

1. Visit the **root node**.
2. Traverse the **left subtree**.
3. Traverse the **right subtree**.

Recursive Algorithm:

```
void preorder(Node* root) {
    if (root != NULL) {
        cout << root->data << " ";
        preorder(root->left);
        preorder(root->right);
    }
}
```

Example Tree:

Traversal Steps:

```
Root → Left → Right:
1 → 2 → 4 → 5 → 3
```

Output: `1 2 4 5 3`

Use Case: Used for **creating a copy** of a tree or **expression tree prefix notation**.

3. Postorder Traversal (Left → Right → Root)

Algorithm Steps:

1. Traverse the **left subtree**.
2. Traverse the **right subtree**.
3. Visit the **root node**.

Recursive Algorithm:

```
void postorder(Node* root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        cout << root->data << " ";
    }
}
```

Example Tree:

Traversal Steps:

```
Left → Right → Root:
4 → 5 → 2 → 3 → 1
```

Output: `4 5 2 3 1`

Use Case: Used for **deleting a tree** or **postfix expression evaluation**.

(B) Breadth First Traversal (BFS)

Breadth First Traversal is also called **Level Order Traversal**.
It visits all nodes **level by level**, starting from the root.

It uses a **queue** data structure.

Algorithm Steps:

1. Create an empty **queue**.
2. Enqueue the root node.
3. While queue is not empty:
    - Dequeue a node.
    - Print its value.
    - Enqueue its left and right children (if they exist).

Time and Space Complexity

Traversal	Time Complexity	Space Complexity
Inorder	O(n)	O(h) (recursion stack)
Preorder	O(n)	O(h)
Postorder	O(n)	O(h)
Level Order	O(n)	O(w) (width of tree)

Where:

- **n** = number of nodes
- **h** = height of tree
- **w** = maximum number of nodes at any level

Example

```
#include <iostream>
#include <queue>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;
};

Node* newNode(int data) {
    Node* node = new Node();
    node->data = data;
    node->left = node->right = NULL;
    return node;
}

void inorder(Node* root) {
    if (root == NULL) return;
    inorder(root->left);
    cout << root->data << " ";
```

```
    inorder(root->right);
}

void preorder(Node* root) {
    if (root == NULL) return;
    cout << root->data << " ";
    preorder(root->left);
    preorder(root->right);
}

void postorder(Node* root) {
    if (root == NULL) return;
    postorder(root->left);
    postorder(root->right);
    cout << root->data << " ";
}

void levelOrder(Node* root) {
    if (root == NULL) return;
    queue<Node*> q;
    q.push(root);
    while (!q.empty()) {
        Node* temp = q.front();
        q.pop();
        cout << temp->data << " ";
        if (temp->left) q.push(temp->left);
        if (temp->right) q.push(temp->right);
    }
}

int main() {
    Node* root = newNode(1);
    root->left = newNode(2);
    root->right = newNode(3);
    root->left->left = newNode(4);
    root->left->right = newNode(5);

    cout << "Inorder Traversal: ";
    inorder(root);
    cout << "\nPreorder Traversal: ";
    preorder(root);
    cout << "\nPostorder Traversal: ";
    postorder(root);
    cout << "\nLevel Order Traversal: ";
    levelOrder(root);

    return 0;
}
```

Output:

```
Inorder Traversal: 4 2 5 1 3
Preorder Traversal: 1 2 4 5 3
Postorder Traversal: 4 5 2 3 1
Level Order Traversal: 1 2 3 4 5
```

Applications of Trees

Application	Description
Hierarchical data representation	File system, organization chart
Binary Search Tree	Efficient searching and sorting
Expression Tree	Used in compilers (to evaluate expressions)
Heaps	Used in priority queues
Tries	Used in autocomplete, dictionary words
Syntax Tree	Used by compilers to parse code
Network routing	Spanning tree protocols